Solutions of Assignment #1 part 2, Total points: 60
(Course: CS 401)

Problem 1 (30 points): Let $G = (V, E)$ be a **directed** graph and let s and t be two nodes of G. Let n and m be the number of nodes and edges of G, respectively. In the class we say how to decide if there is a path **from** s **to** t, namely, we start a (directed) BFS starting from s and check if t appears among the list of nodes that are visited during BFS. The purpose of the assignment is to decide if such a path exists under some **additional constraints**. Let u and v be two other nodes of G that are **not** s or t.

***(i)* [15 points]** Decide if G has a path from s and t that **avoids** using **both** the nodes u and v.

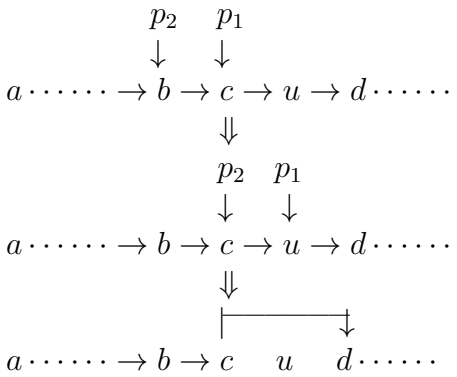
***(ii)* [15 points]** Decide if G has a path from s and t that **uses both** the nodes u and v.

Both of your algorithms should run in $O(m + n)$ time. You may assume that the graph is given in its adjacency list representation. If you are using BFS, there is no need to give codes for it; simply saying "do a BFS starting at such-and-such node" will suffice.

Solution:

(*i*) We delete all the edges in-coming to nodes u and v, *i. e.*, we delete all the edges of the form (x, u) or (x, v) where $x \in V - \{u, v\}$. Then we simply run a BFS starting at s and check if t can be reached.

To delete all the edges as mentioned above, we go through the adjacency list of every node except u and v. For each such list, we traverse the list keeping two pointers, pointer p_1 one at the current entry and pointer p_2 at the entry before the current entry. If the current entry is u or v, we change the link of entry to p_2 to skip u or v. Pictorially, it looks like as shown below for the adjacency list of node a:



(*ii*) We will use the notation $x \rightsquigarrow y$ to indicated a directed path from node x to node y. A path from s to t that uses both nodes u and v must be one of the following two types depending on whether node u is before or after node v:

(A) $s \rightsquigarrow u \rightsquigarrow v \rightsquigarrow t$

(B) $s \rightsquigarrow v \rightsquigarrow u \rightsquigarrow t$

For (A), we can use a BFS starting at s to find a path $s \rightsquigarrow u$, a BFS starting at u to find a path $u \rightsquigarrow v$, and a BFS starting at v to find a path $v \rightsquigarrow t$. (B) can be handled in a similar manner.

Problem 2 (30 points): Suppose that an n-node **directed** graph $G = (V, E)$ contains two nodes u and v such that the shortest path **from** u **to** v contains at least $\frac{3n}{5}$ nodes.

***(i)* [15 points]** Show that there must exist some node x, not equal to either u or v, such that deleting x from G destroys all paths from u to v. (In other words, the graph obtained from G by deleting x contains no directed path from u to v.)

***(ii)* [15 points]** Give an algorithm with running time $O(m + n)$ to find such a node x where m is the number of edges of G. (Hint: use BSF for a directed graph starting at u.)

Solution:

(*i*) We run BFS starting from node u. Let d be the layer in which node v is encountered; by assumption, we have $d \geq \frac{3n}{5}$. We claim first that at least one of the layers $L_1, L_2, \ldots, L_{d-1}$ consists of a single node. Indeed, if each of these layers had at least two nodes, then this would account for at least $2 \times \frac{3n}{5} > n$ nodes; but G has only n nodes.

Thus, there is some layer L_i consisting of just one node, namely the node x as claimed in the problem. We claim next that deleting x destroys all paths from u to v. To see this, consider the set of nodes $X = \{u\} \cup L_1 \cup L_2 \cup \cdots \cup L_{i-1}$. Node u belongs to X but node v does not; and any edge out of X must lie in layer L_i, by the properties of BFS. Since any path from u to v must leave X at some point, it must contain a node in L_i; but x is the only node in L_i.

(*ii*) The BSF in (*i*) can be used to identify the node x in $O(m + n)$ time.